



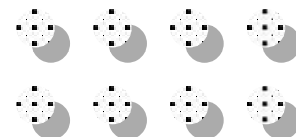
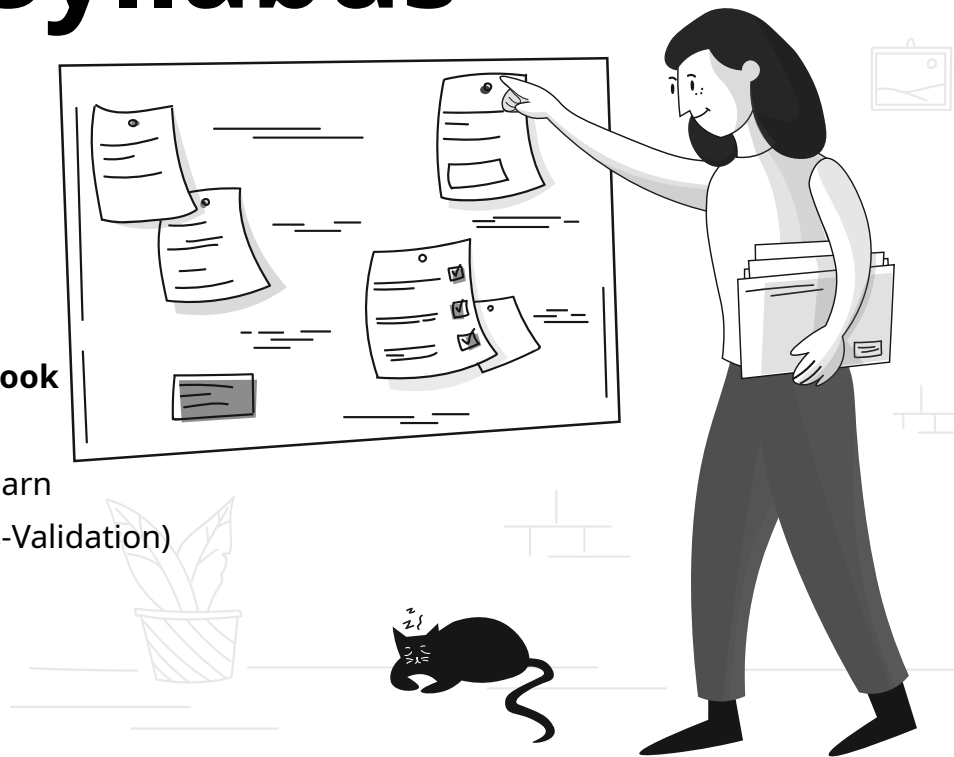
Data Handling & Visualization with Pandas

Eng | Mahmoud Tariq

09/09/25

Course Syllabus

- Intro to Machine Learning & Environment Setup
- Foundations: Key Linear Algebra & Stats + Hands-on NumPy
- **Data Handling & Visualization + Hands-on Pandas**
 - **Task: Solve session 3's 'task.ipynb' notebook**
- The ML Workflow + Linear Regression with Scikit-learn
- Classification Models (Logistic Regression, KNN) with Scikit-learn
- Model Evaluation & Improvement (Metrics, Overfitting, Cross-Validation)
- Mini-Project: Putting it all together with Scikit-learn
- Introduction to Neural Networks
- Building a Neural Network with a Framework
- Course Project Description + Where to Go From Here



Datasets

- What is a Dataset?



- Dataset:

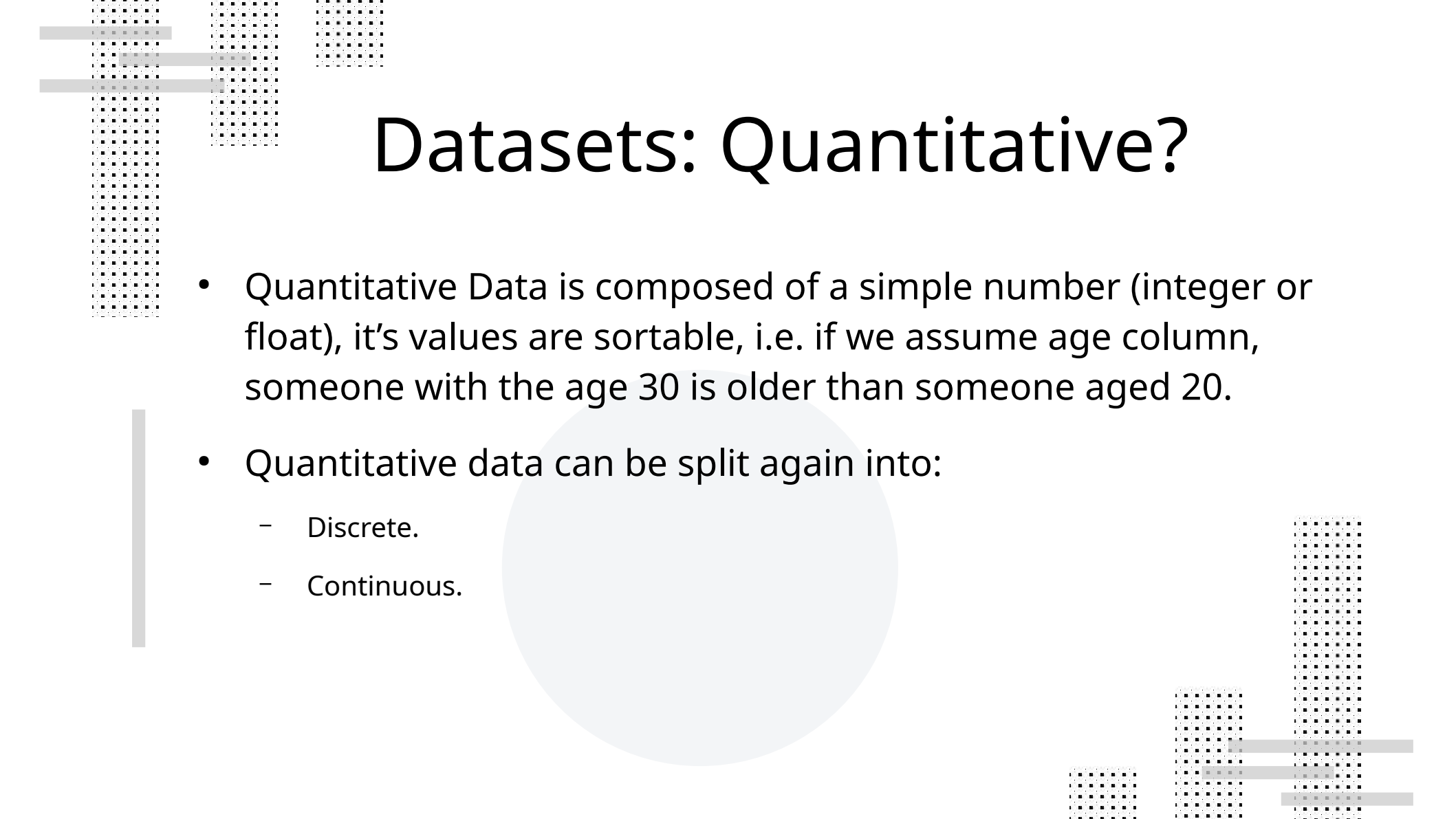
Datasets: instances?

- An instance of data is simply a row in a dataset, and it represents an object with the columns being its attributes.
 - Example: a passenger on the titanic.
 - An example attribute is the 'Name' column.
 - A column with integer values should have all instances with integer values.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S

Datasets: data types?

- Each column of the dataset has a different type, other than whether it's string or int, the type of data can be categorized differently, mainly the types we are talking about are:
 - Quantitative.
 - Qualitative.



Datasets: Quantitative?

- Quantitative Data is composed of a simple number (integer or float), it's values are sortable, i.e. if we assume age column, someone with the age 30 is older than someone aged 20.
- Quantitative data can be split again into:
 - Discrete.
 - Continuous.

Datasets: Quantitative?

- Quantitative discrete data can take any countable or finite numerical value.
 - For example: 3, 256, 154884152 are all discrete.
- Quantitative Continuous data are infinitely imprecise values.
 - For example: if I measured the height of someone, and said he is 1 meter that would be incorrect, cause I can say he is 180 cm, but that is also incorrect because I can say 1802 mm, etc ...
 - Examples: measurements, $\sqrt{2}$, π , etc...

Datasets: Qualitative?

- Qualitative Data can be a number or a string, but it can take only values from a defined set of allowed values.
 - Example: the 'sex' column can only be {'male', 'female'}.
- Qualitative Data can also be split into two categories:
 - Ordinal.
 - Nominal.

Name	Sex	Age	s
Braund, Mr. Owen Harris	male	22.0	
Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	
Heikkinen, Miss. Laina	female	26.0	

Datasets: Qualitative?

- Ordinal Qualitative data has inherent ranking/ordering to it.
 - Example: {'very cold', 'cold', 'neutral', 'hot', 'very hot'} is a set of categories that can be ordered/ranked.
- Nominal Qualitative data has no inherent ranking or ordering.
 - Example: Colors {'blue', 'red', 'green'}, can't be sorted/ranked.

Pandas

- What is Pandas?
- Pandas Hands-on



Pandas?

- Pandas: is a python framework that provides a simple but comprehensive interface for dealing with Datasets.
- You have already installed it in the first session.

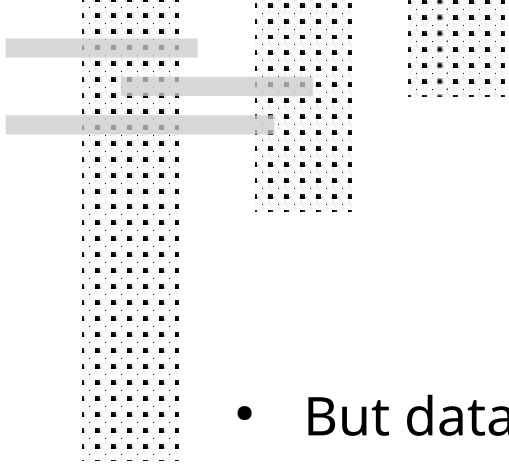
Pandas: Dataframes?

- Pandas represents datasets as 'pd.DataFrame' object which has a lot of useful attributes that we can use to manipulate, clean and visualize data with.
- Creating a Pandas' Dataframe is simple:

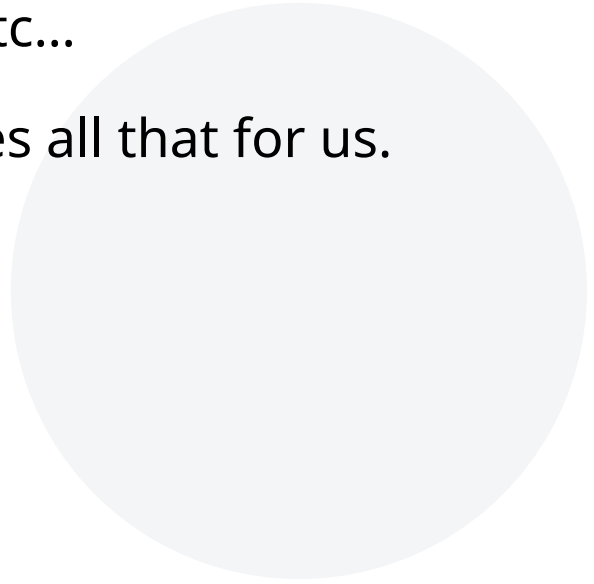
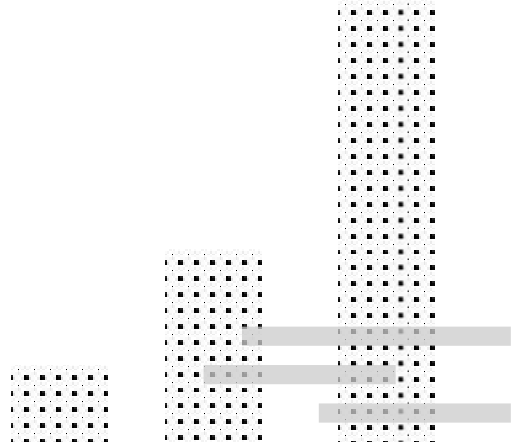
```
fictional_dataset = {'Column A': [1, 2, 3],  
                     'Column B': [4, 5, 6]} # Simple example dataset with two columns 'Column A' & 'Column B'  
  
pd.DataFrame(fictional_dataset)
```

✓ 0.0s

	Column A	Column B
0	1	4
1	2	5
2	3	6



Pandas: Dataframes?

- But datasets don't come in ready to use python dictionaries, data is often messy, saved in their own files with formats like '.csv', '.json', '.xlsx', etc...
 - Pandas handles all that for us.
- 
- 

Pandas: Dataframes?

- In Pandas reading a dataset into the code is very simple.
- We use the 'pd.read_[format]' function, which there are many versions of, e.g. 'pd.read_csv', 'pd.read_json', etc... .
- The one we will be using is 'pd.read_csv' and the dataset is the titanic one we saw at the beginning:

```
import pandas as pd # importing as pd is another standard similar to numpy as np
✓ 0.3s

pd.read_csv("titanic.csv") # the first argument to the function is the path to the dataset
✓ 0.0s
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Pandas: Dataframes?

- 'pd.read_[format]' functions all return a dataframe by default, which is what we will need to handle the data:

```
titanic = pd.read_csv("titanic.csv") # save the dataframe in a variable to use it
titanic
✓ 0.0s
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	NaN	S
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	NaN	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	C
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	Q

891 rows x 12 columns

```
type(titanic)
✓ 0.0s
pandas.core.frame.DataFrame
```


Pandas: Dataframes?

- Let's view some information about the data, first we will view the first n rows with the `'.head()'` function:

```
titanic.head(5) # can be any value, we use 5 to be able to easily view the rows
```

✓ 0.0s

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

- There is also the `'.tail()'` function for viewing the last n rows:

```
titanic.tail(5) # same with the argument in '.head()'
```

✓ 0.0s

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.00	NaN	S
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.00	B42	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.45	NaN	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.00	C148	C
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.75	NaN	Q

Pandas: Dataframes?

- To get the number of total rows & columns we use the `'.shape'` attribute:

```
✓ titanic.shape # returned as (rows, columns)
✓ 0.0s
(891, 12)
```

Pandas: Dataframes?

- To get more information about the columns, their values, and the dataframe as a whole we use '.info()' function:

```
titanic.info()
✓ 0.0s

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    object
4   Sex          891 non-null    object
5   Age          714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    object
9   Fare         891 non-null    float64
10  Cabin        204 non-null    object
11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

Pandas: Dataframes?

- Let's deconstruct it's output:
 - Is the total number of columns.
 - Is the total number of rows & indices.
 - The data type of each column.
 - The number of entries that are not empty.
 - The RAM used by the dataset.
 - The Names of the columns.
 - The number of each data type occurring.

```
titanic.info()
✓ 0.0s

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890 ← 2
Data columns (total 12 columns): ← 1
#   Column      4 Non-Null Count  Dtype  3
---  ---
0   PassengerId 891 non-null    int64
1   Survived    891 non-null    int64
2   Pclass      891 non-null    int64
3   Name        891 non-null    object
4   Sex         891 non-null    object
5   Age         714 non-null    float64
6   SibSp       891 non-null    int64
7   Parch       891 non-null    int64
8   Ticket      891 non-null    object
9   Fare        891 non-null    float64
10  Cabin       204 non-null    object
11  Embarked    889 non-null    object
dtypes: float64(2), int64(5), object(5) ← 7
memory usage: 83.7+ KB ← 5
```

Pandas: Dataframes?

- To get more statistical information about the dataset we use `'.describe()'`:

```
titanic.describe()
```

✓ 0.0s

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
1 count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
2 mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
3 std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
4 min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
5 25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
6 50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
7 75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
8 max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

Pandas: Dataframes?

- Deconstructing it's ouput we get:
 1. The count of values in each column.
 2. The average/mean value for each column.
 3. The standard deviation, i.e. the average value each instance differs by from the mean.
 4. The minimum value for each column.
 5. Indicates that %25 of values land below this one.
 6. Indicates that %50 of values land below this one.
 7. Indicates that %50 of values land below this one.
 8. The maximum value for each column.

```
titanic.describe()
```

✓ 0.0s

	PassengerId	Survived
1 count	891.000000	891.000000
2 mean	446.000000	0.343750
3 std	257.353842	0.477036
4 min	1.000000	0.000000
5 25%	223.500000	0.000000
6 50%	446.000000	0.000000
7 75%	668.500000	1.000000
8 max	891.000000	1.000000

Pandas: Dataframes?

- However, some columns are missing from the `'.describe()'`, mainly any non-numerical type, e.g. `'Name'` column, which is a categorical/qualitative type column, which doesn't define things like a mean, standard deviation, etc...
- but if we go back to the `'.info()'` we can see that it's data type is not `'string'` but `'object'`, this is how Pandas handles strings initially, however we can change that to proper strings.

Pandas: Dataframes?

- First forcing Pandas to include all columns, is done with the 'include' argument:

```
titanic.describe(include='all') # forcing Pandas to include the all columns yields weird & incomplete results
```

✓ 0.0s

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
count	891.000000	891.000000	891.000000	891	891	714.000000	891.000000	891.000000	891	891.000000	204	889
unique	NaN	NaN	NaN	891	2	NaN	NaN	NaN	681	NaN	147	3
top	NaN	NaN	NaN	Braund, Mr. Owen Harris	male	NaN	NaN	NaN	347082	NaN	G6	S
freq	NaN	NaN	NaN	1	577	NaN	NaN	NaN	7	NaN	4	644
mean	446.000000	0.383838	2.308642	NaN	NaN	29.699118	0.523008	0.381594	NaN	32.204208	NaN	NaN
std	257.353842	0.486592	0.836071	NaN	NaN	14.526497	1.102743	0.806057	NaN	49.693429	NaN	NaN
min	1.000000	0.000000	1.000000	NaN	NaN	0.420000	0.000000	0.000000	NaN	0.000000	NaN	NaN
25%	223.500000	0.000000	2.000000	NaN	NaN	20.125000	0.000000	0.000000	NaN	7.910400	NaN	NaN
50%	446.000000	0.000000	3.000000	NaN	NaN	28.000000	0.000000	0.000000	NaN	14.454200	NaN	NaN
75%	668.500000	1.000000	3.000000	NaN	NaN	38.000000	1.000000	0.000000	NaN	31.000000	NaN	NaN
max	891.000000	1.000000	3.000000	NaN	NaN	80.000000	8.000000	6.000000	NaN	512.329200	NaN	NaN

Pandas: Dataframes?

- Including 'object' type columns:

```
titanic.describe(include=['object']) # Only describe type 'object' columns
```

✓ 0.0s

	Name	Sex	Ticket	Cabin	Embarked
count	891	891	891	204	889
unique	891	2	681	147	3
top	Braund, Mr. Owen Harris	male	347082	G6	S
freq	1	577	7	4	644

- This is a correct & valid answer, but why is it type object?

Pandas: Dataframes?

- Pandas deals with a lot of data types, some times even nested json strings.
- So converting them by default to string is removing possibilities.

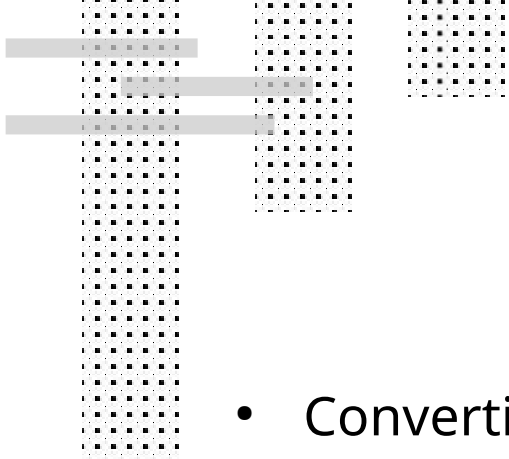
	A	B	Metadata	D
0	sample1_A	sample1_B	{'Data': {'C': {'C_1': 'sample1_C1', 'C_2': 's...	sample1_D
1	sample2_A	sample2_B	{'Data': {'C': {'C_1': 'sample2_C1', 'C_2': 's...	sample2_D

Pandas: Dataframes?

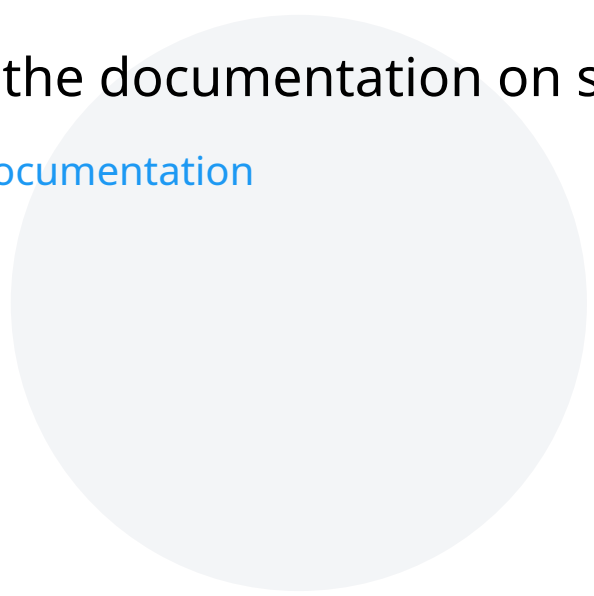
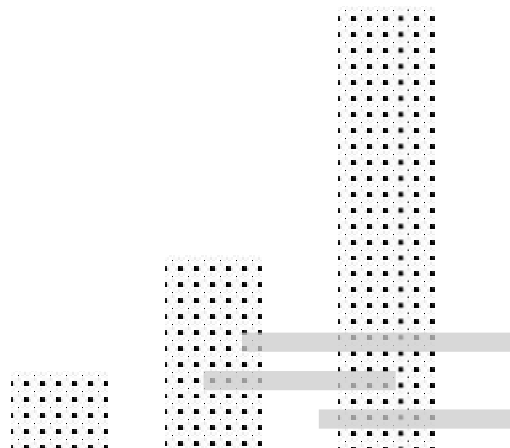
- To convert the 'Name' column for example, we use the 'df['column'].astype(pd.dtype())' function, this function simply casts the values in the column to the passed type:

```
titanic['Name'].astype(pd.StringDtype()) # In this case, we use the pd.StringDtype() dtype
✓ 0.0s

0          Braund, Mr. Owen Harris
1  Cumings, Mrs. John Bradley (Florence Briggs Th...
2          Heikkinen, Miss. Laina
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)
4          Allen, Mr. William Henry
...
886          Montvila, Rev. Juozas
887          Graham, Miss. Margaret Edith
888  Johnston, Miss. Catherine Helen "Carrie"
889          Behr, Mr. Karl Howell
890          Dooley, Mr. Patrick
Name: Name, Length: 891, dtype: string
```



Pandas: Dataframes?

- Converting to strings reveals some useful string functions, but we won't explore them here.
 - You can check the documentation on string functions in Pandas:
 - [Pandas Text Documentation](#)
- 
- 

Pandas: Series?

- 'pd.Series' is another class similar to a dataframe, which houses & allows manipulation of data, however a series only houses one column of data.
- The simplest example is indexing on a column from a dataframe:

```
titanic['Age']  
✓ 0.0s  
  
0      22.0  
1      38.0  
2      26.0  
3      35.0  
4      35.0  
...  
886     27.0  
887     19.0  
888      NaN  
889     26.0  
890     32.0  
Name: Age, Length: 891, dtype: float64  
  
type(titanic['Age'])  
✓ 0.0s  
  
pandas.core.series.Series
```

Pandas: Series?

- Controlling a single column at a time allows us to get more info on the column, for example the counts for each value in a column:

```
titanic['Sex'].value_counts() # this tells us there were 577 males and 314 females.  
✓ 0.0s  
  
Sex  
male      577  
female    314  
Name: count, dtype: int64
```

Pandas: Series?

- Getting the unique values in a column:

```
titanic['Pclass'].unique() # this returns the number of unique passenger/ticket classes  
✓ 0.0s  
array([3, 1, 2])
```

Pandas: Series?

- You can get the full list of methods at:
 - [Pandas' Series Documentation](#)



Pandas: Filtering & Cleaning

- The most important usage and mainly why one would use a framework to handle data is that data is often messy, full of erroneous and missing values.
- Pandas provides a clean and simple interface that allows us to:
 - Filter
 - Clean
 - Fill missing data
 - And more...

Pandas: Filtering

- Firstly we will discuss filtering, to filter data is to extract the part you want to manipulate, and we have been doing so a lot.
- Getting a single/multiple columns from a dataframe is the most basic form of filtering.

Pandas: Filtering

```
titanic['Name'] # Getting a single column is done by simply indexing on the dataframe by it's name. Note: This returns a 'pd.Series' object
```

✓ 0.0s

```
0      Braund, Mr. Owen Harris
1  Cumings, Mrs. John Bradley (Florence Briggs Th...
2      Heikkinen, Miss. Laina
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)
4      Allen, Mr. William Henry
...
886      Montvila, Rev. Juozas
887      Graham, Miss. Margaret Edith
888      Johnston, Miss. Catherine Helen "Carrie"
889      Behr, Mr. Karl Howell
890      Dooley, Mr. Patrick
Name: Name, Length: 891, dtype: object
```

```
columns = ['Name', 'Sex', 'Age'] # a list of column names
```

```
titanic[columns] # to get multiple columns at once together we simply pass a list of the column names as an index. Note: This returns a 'pd.DataFrame' object
# titanic[['Name', 'Sex', 'Age']] passing the list directly without assigning first is also valid and returns the same results
```

✓ 0.0s

	Name	Sex	Age
0	Braund, Mr. Owen Harris	male	22.0
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0
2	Heikkinen, Miss. Laina	female	26.0
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0
4	Allen, Mr. William Henry	male	35.0
...	...	...	...
886	Montvila, Rev. Juozas	male	27.0
887	Graham, Miss. Margaret Edith	female	19.0
888	Johnston, Miss. Catherine Helen "Carrie"	female	NaN
889	Behr, Mr. Karl Howell	male	26.0
890	Dooley, Mr. Patrick	male	32.0

891 rows x 3 columns

Pandas: Filtering

- Getting a single piece of data is done in two ways:
 - `df.loc[index]`
 - `df.iloc[index]`
- The difference between them, is that `'loc[index]'` selects based on the index column, and `'iloc[index]'` is based on relative integer position, i.e. "how many rows down is this row".

Pandas: Filtering

```
titanic.loc[1] # returns the index position at 1
✓ 0.0s

PassengerId      2
Survived         1
Pclass          1
Name      Cumings, Mrs. John Bradley (Florence Briggs Th...
Sex              female
Age           38.0
SibSp          1
Parch          0
Ticket          PC 17599
Fare          71.2833
Cabin          C85
Embarked        C
Name: 1, dtype: object
```

```
titanic.loc[5:10] # returns a pd.DataFrame with the rows with index 5 through to 10 (inclusive)
✓ 0.0s
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8.4583	NaN	Q
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S
7	8	0	3	Paisson, Master. Gosta Leonard	male	2.0	3	1	349909	21.0750	NaN	S
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11.1333	NaN	S
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0	1	0	237736	30.0708	NaN	C
10	11	1	3	Sandstrom, Miss. Marguerite Rut	female	4.0	1	1	PP 9549	16.7000	G6	S

```
titanic.iloc[1] # returns the row with position 1, i.e. one row down in the dataframe.
✓ 0.0s

PassengerId      2
Survived         1
Pclass          1
Name      Cumings, Mrs. John Bradley (Florence Briggs Th...
Sex              female
Age           38.0
SibSp          1
Parch          0
Ticket          PC 17599
Fare          71.2833
Cabin          C85
Embarked        C
Name: 1, dtype: object
```

```
titanic.iloc[5:10] # returns a pd.DataFrame with the rows with positions 5 through to 10 (exclusive), i.e. rows that are down from the top 5, 6.
✓ 0.0s
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8.4583	NaN	Q
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S
7	8	0	3	Paisson, Master. Gosta Leonard	male	2.0	3	1	349909	21.0750	NaN	S
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11.1333	NaN	S
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0	1	0	237736	30.0708	NaN	C

Pandas: Filtering

- No difference? Check this out:

```
reversed_titanic = titanic.iloc[::-1] # this reverses the order of the rows
reversed_titanic.head(5)
```

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
890	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.75	NaN	Q
889	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.00	C148	C
888	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.45	NaN	S
887	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.00	B42	S
886	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.00	NaN	S

```
reversed_titanic.loc[1] # same result
```

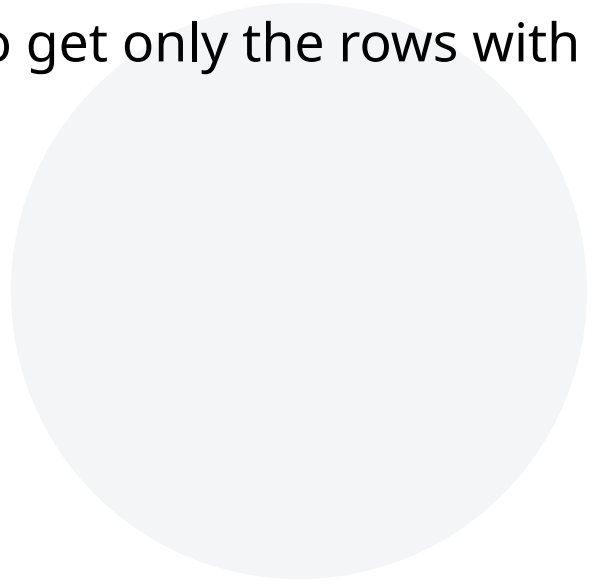
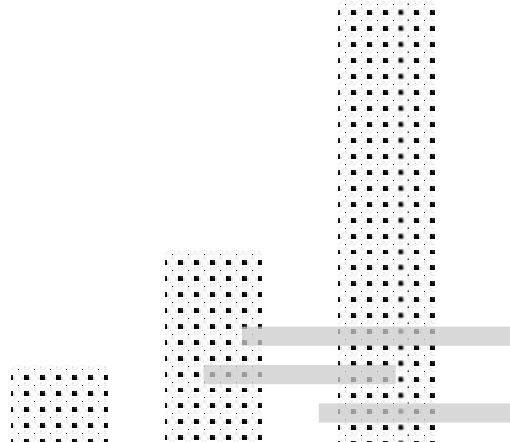
PassengerId	2
Survived	1
Pclass	1
Name	Cummings, Mrs. John Bradley (Florence Briggs Th...
Sex	female
Age	38.0
SibSp	1
Parch	0
Ticket	PC 17599
Fare	71.2833
Cabin	C85
Embarked	C

```
reversed_titanic.iloc[1] # different result, why? '.iloc' simply returns based on "how many rows down" not on index.
```

PassengerId	890
Survived	1
Pclass	1
Name	Behr, Mr. Karl Howell
Sex	male
Age	26.0
SibSp	0
Parch	0
Ticket	111369
Fare	30.0
Cabin	C148
Embarked	C



Pandas: Filtering

- So far we have been filtering by position or column, but what if we want to get rows based on value?
 - For example to get only the rows with the person surviving?
- 
- 

Pandas: Filtering

- This is called masking, and can be done easily, let's follow from our previous example and get those who survived:

```
mask = titanic['Survived'] == 1 # first we select the column we want to compare values on, then we store it in a mask
mask.head() # a mask is a 'pd.Series' object with True and False values
✓ 0.0s

0    False
1     True
2     True
3     True
4    False
Name: Survived, dtype: bool
```

```
titanic[mask] # then we pass the mask as an index to the dataframe
# what pandas does behind the scenes is keeping values where the index in the mask is True and remove them if False
✓ 0.0s
```

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	2	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
8	9	1	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11.1333	NaN	S
9	10	1	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0	1	0	237736	30.0708	NaN	C
...	...	...	...	...	...	...	...	...	...	...	...
875	876	1	Najib, Miss. Adele Kiamie "Jane"	female	15.0	0	0	2667	7.2250	NaN	C
879	880	1	Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)	female	56.0	0	1	11767	83.1583	C50	C
880	881	1	Shelley, Mrs. William (Imanita Parrish Hall)	female	25.0	0	1	230433	26.0000	NaN	S
887	888	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	S
889	890	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	C

342 rows x 12 columns

Pandas: Filtering

- Combining Masks is also possible since they are simply boolean values that we can use “|” and “&” logical operators:

```
survived_mask = titanic['Survived'] == 1
male_mask = titanic['Sex'] == "male"

titanic[survived_mask & male_mask] # this yields rows only where the person is a male and survived
```

✓ 0.0s

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	
17	18	1	2	Williams, Mr. Charles Eugene	male	NaN	0	0	244373	13.0000	NaN	S
21	22	1	2	Beesley, Mr. Lawrence	male	34.0	0	0	248698	13.0000	D56	S
23	24	1	1	Sloper, Mr. William Thompson	male	28.0	0	0	113788	35.5000	A6	S
36	37	1	3	Mamee, Mr. Hanna	male	NaN	0	0	2677	7.2292	NaN	C
55	56	1	1	Woolner, Mr. Hugh	male	NaN	0	0	19947	35.5000	C52	S
...	...	...	...	...	...	...	...	...	...	...	...	...
838	839	1	3	Chip, Mr. Chang	male	32.0	0	0	1601	56.4958	NaN	S
839	840	1	1	Marechal, Mr. Pierre	male	NaN	0	0	11774	29.7000	C47	C
857	858	1	1	Daly, Mr. Peter Denis	male	51.0	0	0	113055	26.5500	E17	S
869	870	1	3	Johnson, Master. Harold Theodor	male	4.0	1	1	347742	11.1333	NaN	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	C

109 rows x 12 columns

Pandas: Filtering

- We can also filter to find missing values (i.e. 'NaN'), but this is built into the series object itself under the '.isna()' method:

```
titanic['Age'].isna() # this returns a mask with rows that have a missing age value
0.0s
0    False
1    False
2    False
3    False
4    False
...
886  False
887  False
888    True
889  False
890  False
Name: Age, Length: 891, dtype: bool

titanic[titanic['Age'].isna()] # this returns rows with missing age values
0.0s
```

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8.4583	NaN	Q
17	18	1	2	Williams, Mr. Charles Eugene	male	NaN	0	0	244373	13.0000	NaN	S
19	20	1	3	Masselmani, Mrs. Fatima	female	NaN	0	0	2649	7.2250	NaN	C
26	27	0	3	Emir, Mr. Farred Chehab	male	NaN	0	0	2631	7.2250	NaN	C
28	29	1	3	O'Dwyer, Miss. Ellen "Nellie"	female	NaN	0	0	330959	7.8792	NaN	Q
...	...	...	...	...	...	...	...	...	...	...	...	
859	860	0	3	Razi, Mr. Raihed	male	NaN	0	0	2629	7.2292	NaN	C
863	864	0	3	Sage, Miss. Dorothy Edith "Dolly"	female	NaN	8	2	CA. 2343	69.5500	NaN	S
868	869	0	3	van Melkebeke, Mr. Philemon	male	NaN	0	0	345777	9.5000	NaN	S
878	879	0	3	Laleff, Mr. Kristo	male	NaN	0	0	349217	7.8958	NaN	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	NaN	S

177 rows x 12 columns

Pandas: Cleaning

- We found the rows that have missing data, but how do we deal with them? How do we “clean” them?
- We have two options:
 - Fill them in with generated values, e.g. average, most common value, etc...
 - Delete them.
- And Pandas has two methods to help us with both ways.

Pandas: Cleaning

- The most simple and easiest approach is dropping/deleting the rows with missing values, this is done via `'.dropna()'` method on both dataframes and series objects:

```
titanic['Age'].dropna() # this removed 178 rows!
✓ 0.0s
```

0	22.0
1	38.0
2	26.0
3	35.0
4	35.0
...	...
885	39.0
886	27.0
887	19.0
889	26.0
890	32.0

Name: Age, Length: 714, dtype: float64

Pandas: Cleaning

- If we actually check the dataframe again the values wouldn't be actually removed, we need to pass an argument 'inplace' for it to take effect or reassign the column:

```
# titanic['Age'].dropna(inplace=True) # we won't be removing values yet
```

Pandas: Cleaning

- Generally we don't remove rows from datasets unless we absolutely need to, what we try first however is filling in the missing values with logical things that are available to us, like:
 - The average value, i.e. the mean (this is used on numerical columns).
 - The most common value, i.e. the mode (this is used on strings/objects).
 - A good known default value, changes based on context and the problem at hand.

Pandas: Cleaning

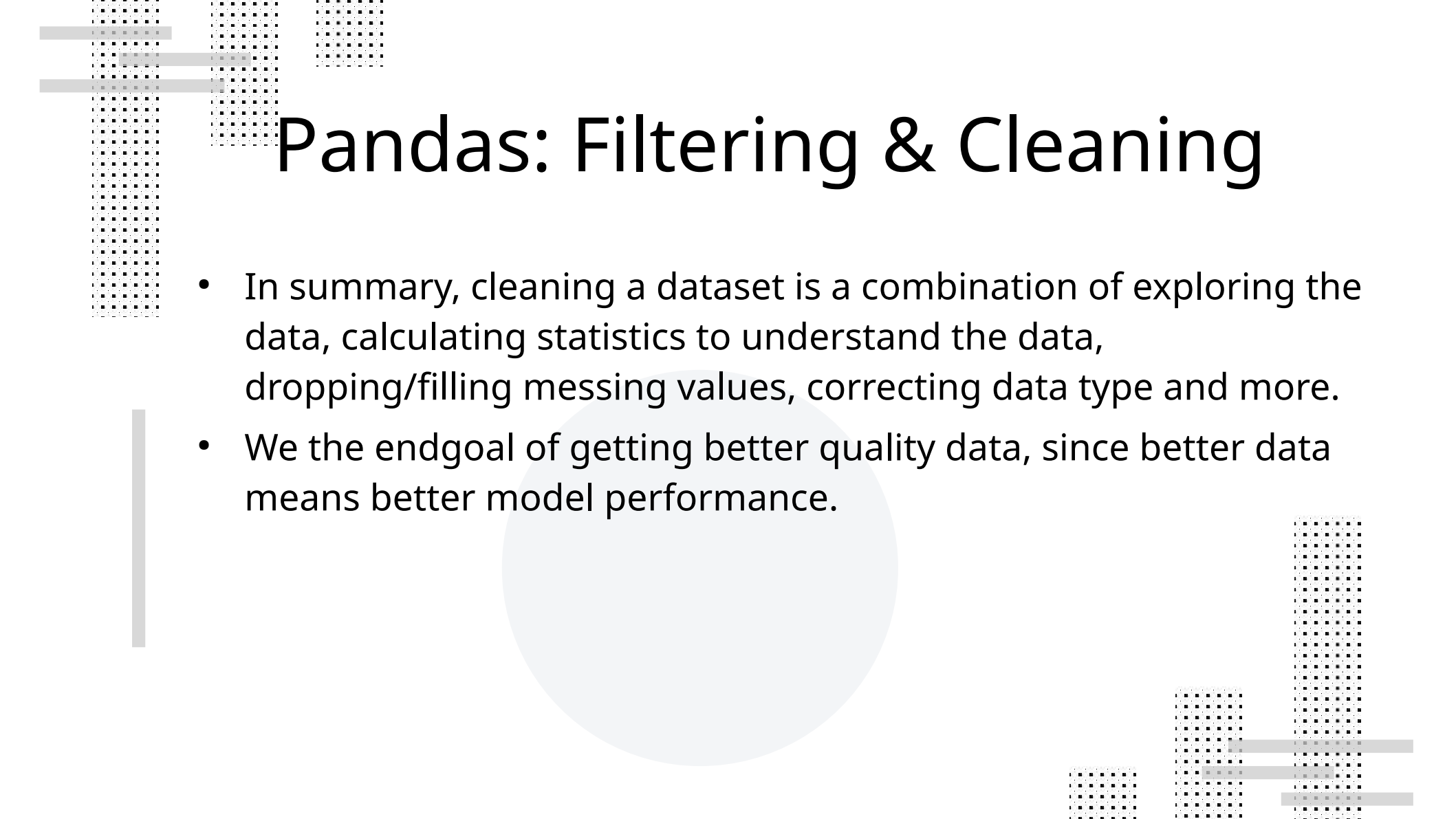
- To fill missing values we use `'.fillna()'` and pass the value to fill with as the first argument:

```
average_age = titanic['Age'].mean() # this calculates the average age
titanic['Age'].fillna(average_age) # this returns a 'pd.Series' object with the 'NaN' values filled
```

✓ 0.0s

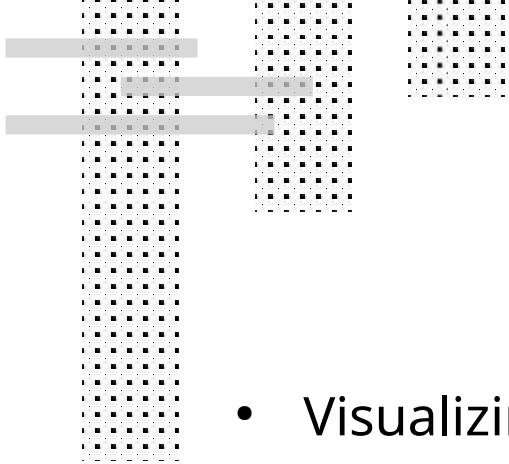
0	22.000000
1	38.000000
2	26.000000
3	35.000000
4	35.000000
...	...
886	27.000000
887	19.000000
888	29.699118
889	26.000000
890	32.000000

Name: Age, Length: 891, dtype: float64

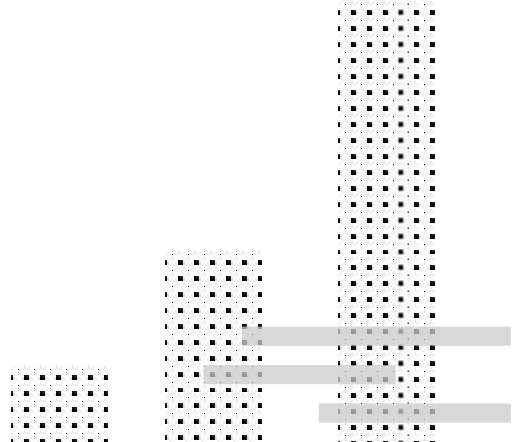


Pandas: Filtering & Cleaning

- In summary, cleaning a dataset is a combination of exploring the data, calculating statistics to understand the data, dropping/filling missing values, correcting data type and more.
- We the endgoal of getting better quality data, since better data means better model performance.

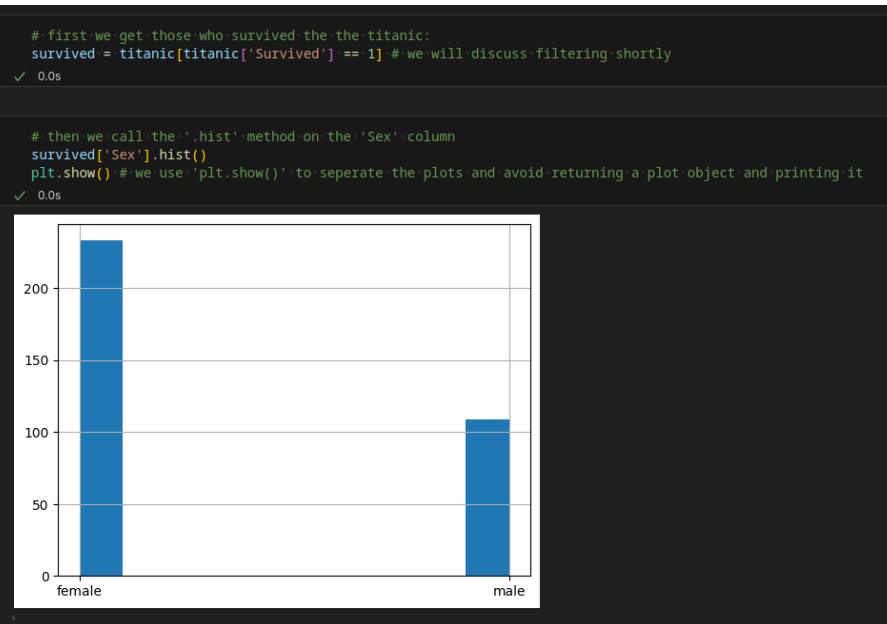


Pandas: Visualization

- Visualizing data is essential for getting a deeper understanding of the data.
 - Pandas and Matplotlib provide a simple interface for displaying info about the data at hand.
- 
- 

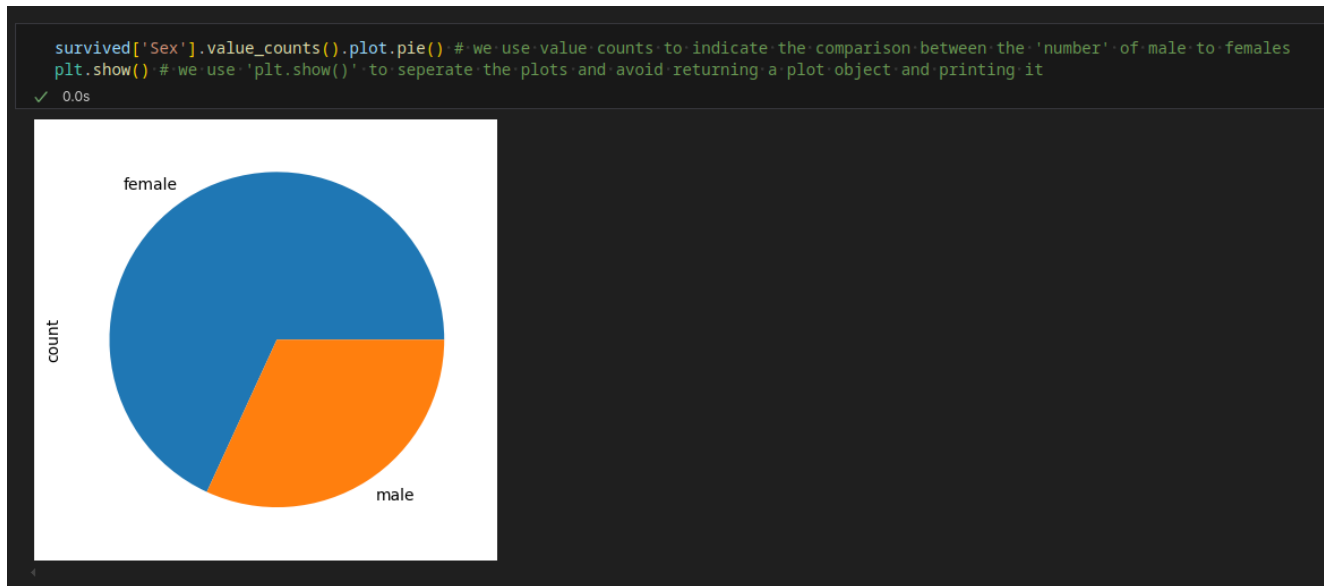
Pandas: Visualization

- First let's explore the `hist()` method, which draws a simple histogram using matplotlib, we will plot survival rate by gender:



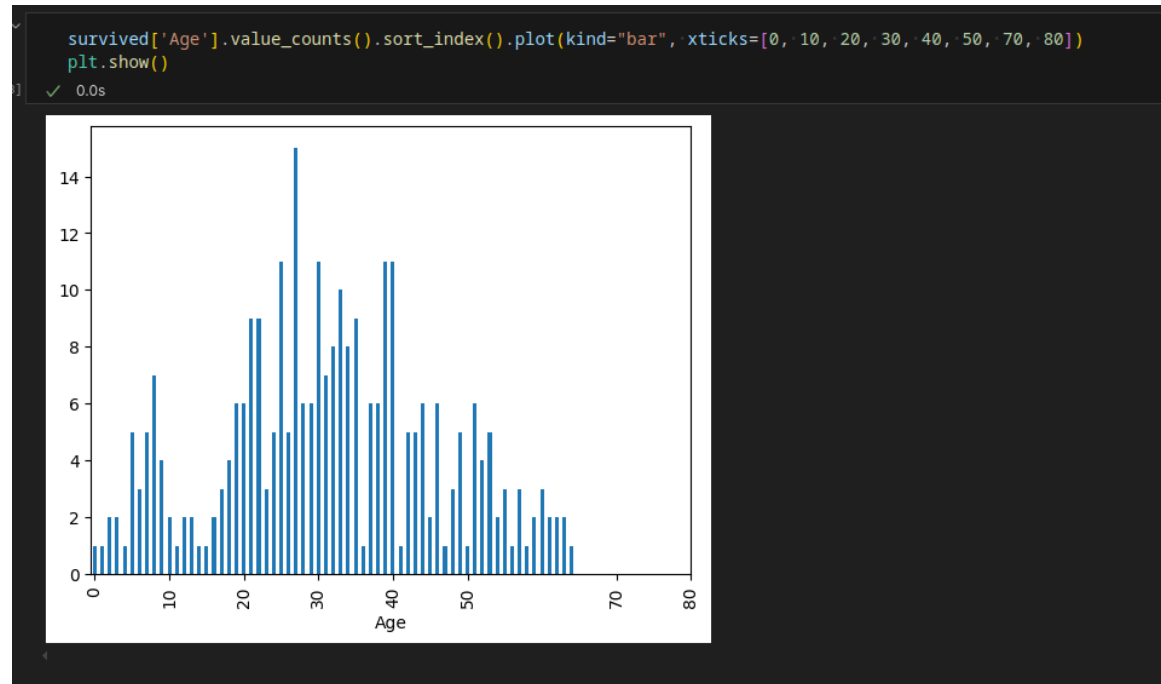
Pandas: Visualization

- Another popular chart is a pie chart, here is how to make it:



Pandas: Visualization

- A bar graph:

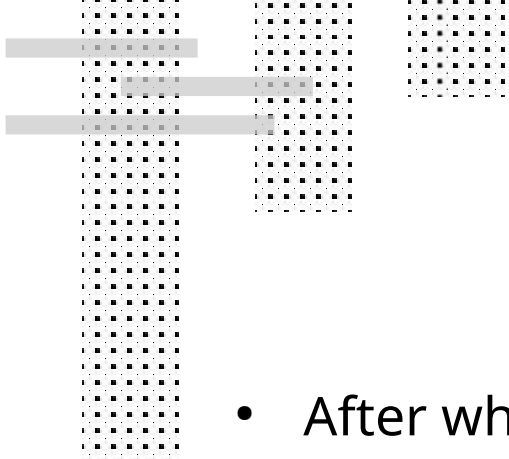


Pandas: Visualization

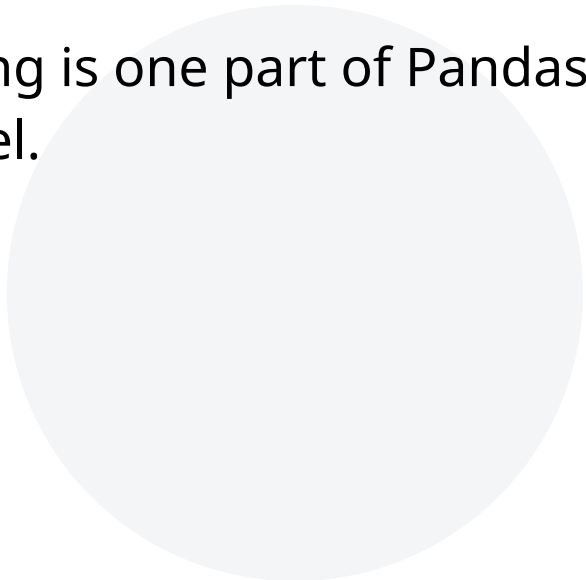
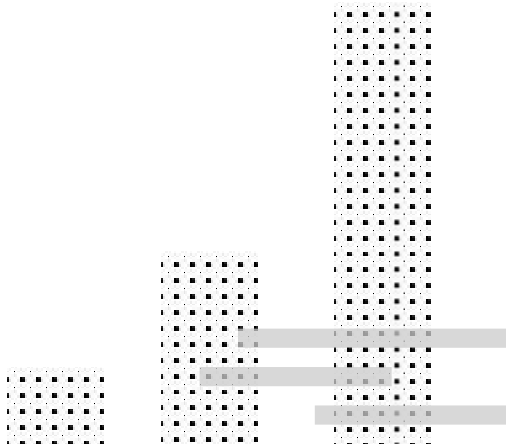
- Since there are too many graphs/charts, you can explore the rest in the documentation:
 - [Pandas Dataframe Plotting documentation](#)

ML & Pandas





Pandas: ML

- After what we discussed we can conclude that pandas is an essential part of any ML engineer's workflow.
 - Clean & Filtering is one part of Pandas' job, the next is feeding data to a model.
- 
- 

Pandas: ML

- As we discussed in the previous session a model is generally in the mathematical form of: $y = X@W + b$.
- The 'y' being the prediction or the wanted outcome.
- The 'X' being the input data, what the model will base it's prediction on.
- "W" & "b" are parts of the model and will be explored later.

Pandas: ML

- Here is an exmple, we will use the titanic dataset for our example, we want to feed the model information about the passenger, and predict if they survived or not.
- This can benefit us by:
 - If the model has a high accuracy, it means that the info we have about the passenger led eventually to their demise.
 - If not, this means that the data we gathered doesn't mean much to the survival of the passenger.

Pandas: ML

- Here is an example code:

```
cleaned_titanic = pd.read_csv("titanic.csv") # We will assume that we already cleaned and saved the data.

X = cleaned_titanic[['Pclass', 'Sex', 'Age', 'Cabin']] # We choose the columns we think are useful & meaningful.

y = cleaned_titanic['Survived'] # The column/information we want to predict later on.

X.shape, y.shape # Check the length of the data to ensure they are equal.
✓ 0.0s

((891, 4), (891,))

# model.fit(X, y) # we 'fit' / train the model on the data to map from input "X" to output "y".
```

Session Task & What's Next

- Your Task: task.ipynb in Session 3 folder
- Next Session: The ML Workflow + Linear Regression with Scikit-learn



Thank you.

